

Getting Started *with* CAN Networking in Linux

This article is aimed at giving the reader a glimpse of the CAN protocol support in the Linux environment through the SocketCAN stack.



The CAN (Controller Area Network) is a standard serial communication protocol that is used to establish communication between various ECUs (electronic control units) inside a vehicle. The current intensive use of electronics within vehicles, the complex network topologies, the need for better wiring harnesses and for reliable transfer of critical data, as well as the ever increasing speed and bandwidth requirements are a few reasons why we need a robust serial communication protocol in the automobile industry.

The CAN protocol offers a multi-master and message-oriented communication architecture in which messages are identified and prioritised using message identifiers. This protocol also incorporates mechanisms such as bitwise arbitration and robust error handling techniques, which assure reliable communication. 11898-1:2015 is the latest standard specification for the CAN 2.0 protocol.

CAN frames

CAN communication occurs in terms of frames. There are four types of frames supported by the CAN protocol.

- **Data frame:** To transmit regular payloads across the nodes
- **Remote frame:** To request the data frame from other nodes (a kind of request-response model)
- **Error frame:** To transmit fault detection in frames
- **Overload frame:** To communicate to the network when a node is busy processing incoming frames

At the outset, a data frame will have the following fields.

- **ID:** 11 bits field for standard and 29 bits for extended frames
 - **Data:** 0 to 8 bytes of payload
 - **Data Length Code (DLC):** Number of bytes present in the data
- CAN FD (flexible data rate), an extension of the original CAN protocol, supports increased bandwidth with payloads up to 64 bytes.